

TEST CASE DOCUMENT

Postman Collection to Pytest Migrator

Project Name: Postman Collection to Pytest Migrator

Document Version: 1.0

1. Introduction

1.1 Purpose

The purpose of this document is to validate the functionality of the Postman Collection to Pytest Migrator. The system converts Postman Collection JSON files into executable Pytest test scripts while preserving request configurations, variables, headers, payloads, query parameters, and assertions.

1.2 Scope

This testing covers:

- Postman Collection Parsing
- Variable Extraction
- URL Construction
- Header Migration
- Query Parameter Migration
- Request Body Migration
- GET Request Conversion
- POST Request Conversion
- PUT Request Conversion
- PATCH Request Conversion
- DELETE Request Conversion
- Assertion Migration
- Pytest Script Generation
- Error Handling

1.3 Objectives

- Verify accurate parsing of Postman collections.
- Validate request migration to Pytest.
- Ensure assertions are converted correctly.
- Confirm generated code follows Python and Pytest standards.
- Validate support for dynamic variables and nested JSON payloads.

2. Test Environment

Parameter	Value
Operating System	Windows 11
Programming Language	Python 3.11
Framework	Pytest
IDE	Visual Studio Code
Input Format	Postman Collection JSON
Output Format	Pytest Python Script
HTTP Library	Requests

3. Test Data

Input Collection Variables

```
{
  "base_url": "https://api.example.com/v1",
  "user_id": "101",
  "token": "sample_jwt_token_123"
}
```

4. Test Cases

TC_001 – Collection Metadata Parsing

Objective

Validate extraction of collection metadata.

Input

```
{
  "info": {
```

```
    "name": "Postman Migrator Full Test Suite",  
    "description": "Collection to validate Postman to Pytest migration."  
  }  
}
```

Test Steps

1. Upload collection JSON.
2. Execute migration process.
3. Parse metadata section.

Expected Result

- Collection name extracted successfully.
- Collection description extracted successfully.

Actual Result

Metadata parsed successfully.

Status

PASS

TC_002 – Variable Extraction

Objective

Validate extraction of collection variables.

Input

```
{  
  "variable": [  
    {  
      "key": "base_url",  
      "value": "https://api.example.com/v1"  
    },  
    {  
      "key": "user_id",  
      "value": "101"  
    }  
  ]  
}
```

Expected Result

Generated Python variables:

```
base_url = "https://api.example.com/v1"
user_id = "101"
```

Actual Result

Variables extracted successfully.

Status

PASS

TC_003 – GET Request Migration

Objective

Verify migration of GET requests with query parameters and assertions.

Input Postman JSON

```
{
  "method": "GET",
  "url": "{{base_url}}/users/{{user_id}}?expand=profile"
}
```

Input Postman Test Script

```
pm.test("Status code is 200", function () {
  pm.response.to.have.status(200);
});

pm.test("User ID exists", function () {
  pm.expect(data.id).to.eql(101);
});
```

Expected Pytest Output

```
def test_get_user_by_id():
```

```
headers = {  
    "Authorization": f"Bearer {token}"  
}  
  
response = requests.get(  
    f"{base_url}/users/{user_id}",  
    params={"expand": "profile"},  
    headers=headers  
)  
  
assert response.status_code == 200  
  
data = response.json()  
  
assert data["id"] == 101
```

Expected Result

- GET request generated.
- Query parameters migrated.
- Assertions migrated.

Actual Result

Migration completed successfully.

Status

PASS

TC_004 – POST Request Migration

Objective

Verify migration of POST requests containing nested JSON payloads.

Input Postman JSON

```
{  
  "method": "POST",  
  "body": {  
    "raw": "{ \"name\": \"John Doe\", \"email\": \"john@example.com\" }"  
  }  
}
```

Expected Pytest Output

```
payload = {
    "name": "John Doe",
    "email": "john@example.com"
}

response = requests.post(
    f"{base_url}/users",
    json=payload
)

assert response.status_code == 201
```

Expected Result

- Payload converted to Python dictionary.
- POST request generated correctly.

Actual Result

Migration completed successfully.

Status

PASS

TC_005 – PUT Request Migration

Objective

Verify migration of PUT requests.

Input

```
{
    "method": "PUT",
    "url": "{{base_url}}/users/{{user_id}}"
}
```

Expected Result

```
response = requests.put(  
    f"{base_url}/users/{user_id}",  
    json=payload  
)  
  
assert response.status_code == 200
```

Actual Result

PUT request generated successfully.

Status

PASS

TC_006 – PATCH Request Migration

Objective

Verify migration of PATCH requests.

Input

```
{  
    "method": "PATCH",  
    "url": "{{base_url}}/users/{{user_id}}/status"  
}
```

Expected Result

```
response = requests.patch(  
    f"{base_url}/users/{user_id}/status",  
    json=payload  
)
```

Actual Result

PATCH request generated successfully.

Status

PASS

TC_007 – DELETE Request Migration

Objective

Verify migration of DELETE requests.

Input

```
{
  "method": "DELETE",
  "url": "{{base_url}}/users/{{user_id}}"
}
```

Expected Result

```
response = requests.delete(
    f"{{base_url}}/users/{{user_id}}"
)

assert response.status_code == 204
```

Actual Result

DELETE request generated successfully.

Status

PASS

TC_008 – Assertion Migration

Objective

Validate conversion of Postman assertions to Pytest assertions.

Input Postman Assertion

```
pm.response.to.have.status(200);  
pm.expect(data.id).to.eql(101);
```

Expected Pytest Assertion

```
assert response.status_code == 200  
assert data["id"] == 101
```

Actual Result

Assertions migrated successfully.

Status

PASS

5. Test Execution Summary

Test Case ID	Test Scenario	Status
TC_001	Collection Metadata Parsing	PASS
TC_002	Variable Extraction	PASS
TC_003	GET Request Migration	PASS
TC_004	POST Request Migration	PASS
TC_005	PUT Request Migration	PASS
TC_006	PATCH Request Migration	PASS
TC_007	DELETE Request Migration	PASS
TC_008	Assertion Migration	PASS

6. Feature Coverage Matrix

Feature	Coverage Status
Collection Parsing	Covered

Feature	Coverage Status
Variable Handling	Covered
Header Extraction	Covered
URL Construction	Covered
Query Parameters	Covered
GET Requests	Covered
POST Requests	Covered
PUT Requests	Covered
PATCH Requests	Covered
DELETE Requests	Covered
Assertion Migration	Covered
Pytest Generation	Covered

7. Test Result Summary

Metric	Value
Total Test Cases	8
Passed	8
Failed	0
Blocked	0
Success Rate	100%

8. Conclusion

The Postman Collection to Pytest Migrator successfully converts Postman collection requests and associated test scripts into executable Pytest test cases. All supported HTTP methods, variables, request payloads, headers, query parameters, and assertions were migrated accurately. The generated Pytest scripts comply with Python coding standards and are ready for integration into CI/CD pipelines.